

ECON485 Introduction to Database Systems

NoSQL Databases

Introduction

Traditional Relational Database Management Systems (RDBMS), while reliable, face limitations when dealing with the challenges of modern web-scale applications and data complexity.

The challenges were initially discussed with terms always starting with “V” (ie. velocity, volume, etc) so we talk about 4Vs, 5Vs, etc.

The Challenge of Scale (Volume & Velocity)

Massive Data Volume: The amount of data generated by users, sensors, and machines has exploded (Big Data). RDBMS often struggle to manage petabytes of data cost-effectively.

High Velocity (Traffic): Modern applications (e.g., social media, e-commerce) demand millions of reads and writes per second.

The RDBMS Bottleneck (Vertical Scaling):

- RDBMS primarily scale vertically (adding more CPU, RAM to a single, expensive server).
- This approach has a hard limit and becomes prohibitively expensive.

Proposed Solution: Something that embraces Horizontal Scaling (sharding data across many inexpensive commodity servers).

The Challenge of Structure (Variety)

Fixed Schema Rigidity: RDBMS requires a strict, predefined schema enforced by the database.

- Any change (e.g., adding a new field) requires schema migration, which is slow and disrupts high-volume applications.

Unstructured and Semi-structured Data: A huge amount of modern data (user profiles, logs, sensor readings) is unstructured or semi-structured (e.g., JSON, XML).

Required Flexibility: We need databases that offer a dynamic schema (schema-less) approach.

- This allows developers to evolve the application and data model quickly without downtime.
- It is a natural fit for storing diverse data types.

The Challenge of Performance & Availability

Low Latency Requirement: Users expect near-instantaneous responses. Traditional RDBMS joins and complex transactions can introduce significant latency at scale.

High Availability is Non-Negotiable: With a single database server (vertical scaling), a failure means the entire application is down. Modern apps must be always-on.

Required Approach:

- Sacrifice some traditional RDBMS features (like complex joins and strict transaction integrity) for massive throughput and lower latency.
- By distributing data across multiple nodes (horizontal scaling), a failure of one node does not bring down the whole system, ensuring high availability.

The Challenge of Performance & Availability

Recap: Why NoSQL? NoSQL was born to solve the scale, variety, performance, and availability challenges that RDBMS struggled with.

The Core Decision: The ability to scale horizontally and remain highly available requires a fundamental trade-off in database design.

The Ultimate Constraint: This necessary sacrifice of traditional database guarantees is formally defined by the CAP Theorem.

Transition: The CAP theorem states that a distributed data store can only simultaneously guarantee two of the following three properties:

- Consistency
- Availability
- Partition Tolerance

Recap

Volume: The sheer quantity of data generated (petabytes, zettabytes). RDBMS struggles with single-server scaling limits; NoSQL offers inexpensive horizontal scaling.

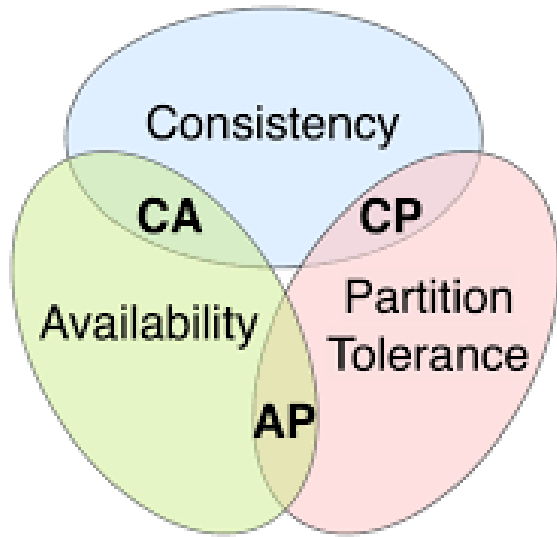
Velocity: The speed at which data is generated, collected, and needs to be processed. RDBMS transactions and locking can be too slow; NoSQL is optimized for high-speed reads/writes (throughput).

Variety: The many types of data: structured (tables), semi-structured (JSON), and unstructured (text, video). RDBMS requires rigid schema; NoSQL offers dynamic, flexible schemas to handle diverse data types.

Veracity: The trustworthiness and quality of the data (inconsistency, biases, noise). While not directly a database feature, managing massive, noisy datasets often requires the flexible storage and processing capabilities of NoSQL and Big Data platforms.

Value: The ability to turn the data into meaningful insights that drive decisions. NoSQL databases and their associated ecosystem (e.g., MongoDB Aggregation, Hadoop) provide the foundation necessary to extract value from complex, large datasets.

CAP Theorem



CAP Theorem states:
A distributed data store cannot simultaneously guarantee all three of the following properties during a network partition: Consistency, Availability, and Partition Tolerance.

System designers must choose to prioritize two of the three properties based on application requirements.

CAP Theorem

The Three Properties of the CAP Theorem

- Consistency (C): Every client sees the same data at the same time, regardless of which node they connect to. A read operation is guaranteed to return the most recent write, or an error if that guarantee cannot be met. This is the traditional goal of ACID (Atomicity, Consistency, Isolation, Durability) transactions.
- Availability (A): Every non-failing node in the system must return a response for every request in a reasonable amount of time. The response is not guaranteed to be the most recent data, but the system remains operational for all clients. This maximizes uptime and responsiveness to users.

CAP Theorem

The Three Properties of the CAP Theorem

- Partition Tolerance (P): The system continues to operate despite arbitrary network failures (partitions) that disrupt communication between nodes. In any real-world distributed system, network partitions are inevitable, making partition tolerance a necessary characteristic for reliable service.

CAP Theorem

Since network partitions are unavoidable in distributed systems, the practical application of the CAP theorem is a choice between prioritizing consistency or availability during a partition event.

CP: Applications needing strong data accuracy (e.g., banking systems, inventory). CP focused databases like MongoDB and Redis prioritize **consistency**.

- When a partition occurs, the system may shut down inconsistent nodes (making them unavailable) until data can be synchronized.
- How? When a network partition occurs, the system will stop serving requests (making it unavailable) on the "minority" side of the partition to ensure that data conflicts cannot occur.
- Why? In systems where data loss or conflicting writes are unacceptable (banking ledgers), being unavailable is the better choice.

CAP Theorem

Since network partitions are unavoidable in distributed systems, the practical application of the CAP theorem is a choice between prioritizing consistency or availability during a partition event.

AP: Applications needing continuous uptime (e.g., social media feeds, online retail browsing). Databases such as Apache Cassandra and Amazon DynamoDB prioritize availability.

- During a partition, all nodes remain active and accept requests, though some may return slightly outdated data (known as "eventual consistency").
- How? When a network partition occurs, the system continues to serve requests on both sides of the partition. This means the data seen by two different clients might be temporarily different.
- Why? On systems where high uptime is critical and temporary inconsistencies are acceptable (social media feeds, e-commerce product catalogs, etc) this is the better choice.

CAP Theorem

CA: Theoretically possible only in systems with perfectly reliable networks and no partitions.

RDBMS (ie. SQL databases) were theorized before CAP theorem was out there. All SQL databases are initially CA because they are assumed to be a single “node” deployment. When they scale, problems arise.

Three Major NoSQL Database Types

Key-Value Stores

- CAP Theorem fit: Typically AP (Availability + Partition Tolerance), often with eventual consistency. Examples like Redis (in-memory, can be configured) and DynamoDB prioritize high availability and scalability over strict consistency.
- General usefulness: Simplest and fastest for high-performance lookups, caching, session storage, shopping carts, or any scenario needing quick access by a unique key. Ideal for large-scale reads/writes where complex queries aren't needed.
- Major examples: Redis, Amazon DynamoDB, Riak.

Three Major NoSQL Database Types

Operations equivalents:

- Insert: PUT(key, value) or SET(key, value) – Adds a new key-value pair (opaque value treated as a blob).
- Select (read): GET(key) – Retrieves the value by exact key (no querying beyond key).
- Update: Same as insert – PUT or SET overwrites the value if the key exists (no separate update command; it's upsert-like).

Three Major NoSQL Database Types

Document Stores

- CAP Theorem fit: Varies; MongoDB is often tunable but defaults to CP (Consistency + Partition Tolerance) in some configurations, while others like CouchDB are AP with eventual consistency.
- General usefulness: Flexible schema for semi-structured data (JSON/BSON-like documents). Great for content management, user profiles, catalogs, or applications with evolving data models needing nested structures and queries on fields.
- Major examples: MongoDB, CouchDB, Couchbase.

Three Major NoSQL Database Types

Operations equivalents:

- Insert: `insertOne()` or `insertMany()` – Adds new document(s); auto-generates `_id` if not provided.
- Select (read): `find()` or `findOne()` – Queries documents by criteria (e.g., field matches, ranges); supports aggregation pipelines for complex processing.
- Update: `updateOne()`, `updateMany()`, or `replaceOne()` – Modifies existing documents; supports operators like `$set` for specific fields. Upsert option (`upsert: true`) inserts if no match.

Three Major NoSQL Database Types

Column-Family (Wide-Column) Stores

- CAP Theorem fit: Typically AP (e.g., Cassandra prioritizes availability and partition tolerance with tunable consistency, often eventual).
- General usefulness: Handles massive datasets with sparse, wide rows (billions of columns). Suited for time-series data, analytics, logging, or high-write throughput scenarios like IoT or recommendation engines.
- Major examples: Apache Cassandra, HBase, ScyllaDB.

Three Major NoSQL Database Types

Operations equivalents (using CQL in Cassandra):

- Insert: `INSERT INTO table ... VALUES ...` – Adds a new row or columns.
- Select (read): `SELECT ... FROM table WHERE ...` – Queries by primary key (partition + clustering); limited secondary indexes.
- Update: `UPDATE table SET ... WHERE ...` – Identical semantics to `INSERT` (upsert: overwrites existing columns or creates row if absent).

Vector Databases

Vector databases are not considered a fourth major NoSQL type alongside key-value, document, and column-family stores. They are a specialized category optimized for storing and querying high-dimensional vector embeddings (numerical representations of data like text, images, or audio from ML models).

Their primary operation is similarity search (e.g., nearest neighbors via cosine similarity or Euclidean distance), often using indexes like HNSW or IVF.

Vector Databases

Many dedicated vector DBs (e.g., Pinecone, Milvus, Weaviate) are built on or resemble NoSQL architectures (often document-like with metadata), prioritizing scalability and approximate nearest neighbor (ANN) search.

They inherit CAP properties from their underlying design (many are AP for distributed setups), but no unique "vector CAP" exists beyond the standard theorem—trade-offs remain consistency vs. availability during partitions.

A proposed "new CAP" for vectors (Cost, Accuracy, Performance) is informal and focuses on indexing trade-offs, not the distributed systems theorem.

Vector Databases

Some traditional SQL databases with vector extensions:

- PostgreSQL (pgvector extension): Adds a vector data type to columns. You store embeddings as arrays (e.g., [0.1, 0.2, ...] up to 2000+ dimensions). It supports similarity operators (<-> for Euclidean, <=> for cosine) and indexes (e.g., IVFFlat or HNSW for faster ANN search). Queries integrate seamlessly with SQL (e.g., `SELECT * ORDER BY embedding <-> '[query_vector]' LIMIT k;`). No separate DB needed—leverages Postgres ACID, joins, and filtering.
- Oracle (AI Vector Search in 23ai+): Native VECTOR data type with formats (dense/sparse, int/float). Supports vector indexes and SQL extensions for similarity search. Integrates with relational data for hybrid queries (e.g., combine vector similarity with business filters). Works alongside JSON, graph, etc., in the same database.

Use More Than One DB Type Together?

This approach, known as polyglot persistence, involves using multiple database technologies tailored to specific data types, workloads, or features, rather than forcing everything into a single system.

RDBMS still excel at structured data, ACID transactions, complex joins, and relational integrity—ideal for core business logic like user accounts, orders, billing, or inventory.

NoSQL databases (key-value, document, column-family, or graph) shine in scenarios where RDBMS fall short.

Use More Than One DB Type Together?

If your app starts simple, begin with RDBMS and add NoSQL later when pain points emerge (e.g., scale or schema changes).

- Handling semi-structured or unstructured data (e.g., JSON documents for user profiles or product catalogs).
- High-write throughput or massive scale (e.g., logging, session storage, real-time analytics, or IoT data).
- Flexible schemas for rapidly evolving applications.
- Horizontal scaling at lower cost.

Use More Than One DB Type Together?

Common use cases for hybrid RDBMS + NoSQL:

- E-commerce: RDBMS for transactions and relationships; document store (e.g., MongoDB) for product details or user-generated content; key-value (e.g., Redis) for caching/session management.
- Analytics-heavy apps: RDBMS for operational data; column-family (e.g., Cassandra) for time-series or big data logs.
- Large enterprises: Many real-world systems (e.g., Netflix, Amazon) use polyglot persistence to optimize performance and cost.

Use More Than One DB Type Together?

Pros:

- Better performance and scalability for diverse workloads.
- Cost efficiency (horizontal scaling vs. expensive vertical upgrades).
- Flexibility for agile development.

Cons:

- Increased complexity (data synchronization, consistency across stores, more DevOps overhead).
- Potential for eventual consistency issues (vs. RDBMS ACID guarantees).

Use More Than One DB Type Together?

How about vector databases?

Vector databases (dedicated like Pinecone, Milvus, Weaviate; or extensions like pgvector in PostgreSQL) are optimized for storing and querying high-dimensional vector embeddings—essential for AI/ML features like similarity search, recommendations, or semantic queries.

RDBMS with vector extensions (e.g., PostgreSQL's pgvector, Oracle's AI Vector Search) can handle basic vector needs integrated with relational data, but dedicated vector DBs excel at large-scale approximate nearest neighbor (ANN) searches.

Use More Than One DB Type Together?

Common use cases for hybrid RDBMS + Vector:

- RAG (Retrieval-Augmented Generation) in LLMs: Store embeddings in vector DB; link to metadata/structured data in RDBMS for hybrid queries (e.g., "find similar products under \$50 from brand X").
- Recommendation engines: Vectors for similarity; RDBMS for user/transaction history.
- Image/text search: Vectors for semantic matching; RDBMS for filtering by exact attributes (price, date, category).

Use More Than One DB Type Together?

Pros:

- Enables advanced AI features (e.g., semantic search, anomaly detection) without rebuilding your core database.
- Hybrid search: Combine vector similarity with precise SQL filters/joins.
- Scalability for growing embedding volumes.

Cons:

- Added integration effort (e.g., syncing embeddings with source data).
- If needs are modest, start with RDBMS extensions to avoid extra infrastructure.

Thanks